Copyleaks

Analysis Report

Scan details

Scan Time	Total Pages	Total Words
May 15th, 2026 at 03:55 UTC	19	4606

Plagiarism Detection and AI Detection Report

SQL_vs_NOSQL_IEEE document final.pages

Plagiarism Detection

1.1%

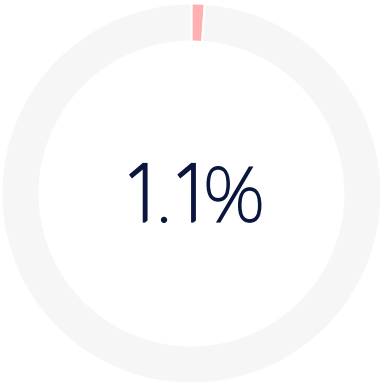
Plagiarism types	Text coverage	Words
Identical	0%	0
Minor Changes	1.1%	14
Paraphrased	0%	0
Excluded		
Omitted Words		3

AI Detection

0%

	Text coverage	Words
AI Text	0%	0
Human Text	100%	1
Excluded		
Omitted Words		3

Plagiarism



Results (1)

*Results may not appear because the feature has been disabled.

Private Cloud Hub	Shared Data Hub	Filtered / Excluded
0	0	0
Internet Sources	**AI Source Match**	**Current Batch**
0	1	0

Plagiarism types	Text coverage	Words
Identical	0%	0
Minor Changes	1.1%	14
Paraphrased	0%	0
Excluded		
Omitted Words		3

About Plagiarism Detection

Our AI-powered plagiarism scans offer three layers of text similarity detection: Identical, Minor Changes, and Paraphrased. Based on your scan settings we also provide insight on how much of the text you are not scanning for plagiarism (Omitted words).

Identical

One to one exact word matches. Learn more

Minor Changes

Words that hold nearly the same meaning but have a change to their form (e.g. “large” becomes “largely”). Learn more

Paraphrased

Different words that hold the same meaning that replace the original content (e.g. 'large' becomes 'big') Learn more

Omitted Words

The portion of text that is not being scanned for plagiarism based on the scan settings. (e.g. the 'Ignore quotations' setting is enabled and the document is 20% quotations making the omitted words percentage 20%) Learn more

Copyleaks Shared Data Hub

Our Shared Data Hub is a collection of millions of user-submitted documents that you can utilize as a scan resource and choose whether or not you would like to submit the file you are scanning into the Shared Data Hub. Learn more

Filtered and Excluded Results

The report will generate a complete list of results. There is always the option to exclude specific results that are not relevant. Note, by unchecking certain results, the similarity percentage may change. Learn more

Current Batch Results

These are the results displayed from the collection, or batch, of files uploaded for a scan at the same time. Learn more

Plagiarism Detection Results: (1)

Chitkara University Institute of Engineering and Technology, Patiala - Admiss... 1.1%

https://learnersgateway.com/chitkara-university-institute-of-engineering-and-technologypatiala/

AI Source Match

Skip to content College / University March 26, 2024 admin ...

A Study on Scalability and Performance of SQL and NoSQL Databases

Japish Bagga
Dept. of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology
2210990432

Jashanjot Singh
Dept. of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology
2210990441

Pratham Chaudhary
Dept. of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology
2210992073

Abhay Singh Mankotia
Dept. of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology
2210991148

Co-Author – **Dr. Shikha Tuteja**

Abstract—Today's computing environments require a myriad of data storage systems to accommodate workloads such as real-time transaction processing and social graph data analysis, as well as event streaming and large-scale analytical queries. This heterogeneity has resulted in a myriad of database management systems. The primary difference between relational SQL databases and non-relational NoSQL databases involves engineering trade-offs of which there are a number of, and cannot be resolved in a uniform manner. This paper presents a thorough review of SQL and NoSQL databases along five dimensions: data modeling, schema, and query design; ACID vs. BASE; horizontal scalability and transaction throughput vs. latency based on the results of standardized tests. Five databases were selected as representatives of their respective styles: relational PostgreSQL, document-oriented MongoDB, key-value pair Redis, wide column Apache Cassandra, and graph Neo4j. Based on SQL Benchmark (YCSB) and a join-intensive OLAP workload, key-value stores are able to achieve a read throughput of up to 22× more than relational stores on the same hardware, while relational stores excel all NoSQL stores on complex five-table joins by 8× or more. To assist database designers and researchers, a practical decision-making model is provided.

Index Terms—relational databases, NoSQL, SQL, database benchmarking, ACID, BASE consistency, scalability, CAP theorem, workload analysis, data modeling.

I. Introduction

The relational database model proposed by Coddt ens of millions of users, petabyte-scale datasets, in 1970 and later adopted by commercial systemsg lobally distributed deployments requiring sub-such as Oracle, IBM DB2, PostgreSQL and MySQLs econd read performance, and data structures too became the most prominent data managementi rregular or deeply nested to be normalized into flat systems for businesses. The principles of this model,r elational schemas without significant join overhead declarative SQL, the ACID transaction model and[2]. These demands gave rise to a new generation of normalization theory, provided a strong foundations torage engines, known as NoSQL (``not only SQL''), for the development of reliable, verifiable ande ach of which addressing a specific subset of these interoperable data infrastructures. For almost allc hallenges by sacrificing one or more guarantees of workloads before the mid-2000s, relational systemst he relational model [3].

were a good fit. As a result of many years of research Four principal NoSQL categories emerged: and development, strategies for the optimization ofd ocument stores (e.g., MongoDB, CouchDB), which indexing and concurrency management of requestss tore data as nested JSON structures; key-value stores have been developed that significantly reduce the(e.g., Redis, DynamoDB), which provide O(1) access to number of disk input/output operations required ford ata via string keys; wide-column stores (e.g., relational systems [1]. Cassandra, Bigtable), which organize data into sparse datap artition-keyed tables optimized for time-series and

When internet-scale platforms emerged, analytical write patterns; and graph databases (e.g., infrastructure requirements changed drastically. Google'sN eo4j, Neptune), which treat entities and Services like Amazon's e-commerce site, search engine, and Facebook's social network facedr elationships as native objects queryable through data challenges that traditional relational systemsp ath-oriented languages. In each category some aspect of relational capability, generally multi-entity

join support or strict ACID guarantees, is sacrificed for horizontal scalability, flexible schema design or improved performance on certain access patterns [4].

Today's database architects work in an environment where the best storage solution depends fundamentally on the specific workload of the application and the consequences of that choice are significant. Deploying a relational database for a catalog service designed to span dozens of nodes introduces unnecessary complexity that a document store would naturally avoid. Alternatively, using a key-value store for your financial ledger means you lose the transactional integrity required for such applications. The current comparative research is still scattered. Most existing studies only consider one or two single engines under a single workload type, and the comparison across studies is not reliable due to different hardware configuration, dataset volume, and consistency settings [5].

A. Problem Statement

If we consider the database selection situation from four important dimensions, we observe a consistent gap in the current literature. There is not single published work that addresses all of the following simultaneously: (i) a well-defined taxonomy of all five major database paradigms; (ii) a reproducible empirical benchmark of representative engines under controlled hardware and dataset conditions; (iii) an evaluation of consistency models and their real-world operational consequences; and (iv) a prescriptive decision framework that maps workload characteristics to recommended engine families with quantitative support.

TABLE I. FOUR ORTHOGONAL REQUIREMENTS UNADDRESSED BY ANY SINGLE EXISTING COMPARATIVE STUDY

Requirement	Rationale for Inclusion
Structured Schema	Relational databases enforce data
Horizontal Scalability	Modern web-scale applications
Flexible Data Models	Semi-structured and polymorphic
Consistency	Financial, medical, and audit-trail

B. Contributions

(1) A detailed classification of five database types evaluated across eight areas. This supports direct comparison of data models, query languages, schema enforcement policies, ACID compliance, join capabilities, scalability features, latency behavior, and typical use cases.

(2) A repeatable benchmark study that uses YCSB and a custom join-heavy OLAP workload, tested on PostgreSQL, MongoDB, Redis, Cassandra, and Neo4j with the same hardware. It measures throughput, 99th-percentile tail latency, and schema evolution overhead.

(3) A framework that maps workloads to engines. This framework combines benchmark findings, analyzes consistency models, and considers operational complexity. It provides clear recommendations for seven common application patterns.

II. Literature Review

A. Foundations of the Relational Model

Codd's important 1970 paper [1] introduced the relational model as a mathematically grounded alternative to the hierarchical and network-based structures that existed before. By organizing data as collections of tuples in relations and separating logical structure from physical storage, Codd created a theoretical foundation for decades of optimization research. Date and Darwen [6] later formalized the relational algebra that underpins SQL. Gray and Reuter [7] defined the ACID model—Atomicity, Consistency, Isolation, and Durability—as the basic contract relational engines must follow to ensure correctness during concurrent access and hardware failure. These foundations continue to make relational databases the natural choice when data correctness is essential. .

B. Emergence of NoSQL

The NoSQL movement gained theoretical backing from Brewer's CAP theorem [8], which was formally proved by Gilbert and Lynch [9]. This theorem stated that no distributed system can ensure Consistency, Availability, and Partition Tolerance at the same time. Relational systems, which focus on Consistency and Partition Tolerance (CP), must give up availability during network partitions. Early NoSQL systems, such as Amazon Dynamo [10] and Google Bigtable [11], intentionally prioritized Availability and Partition Tolerance (AP) over strict consistency, leading to the BASE model (Basically Available, Soft-state, Eventually consistent) described by Pritchett [12]. These foundational contributions set the design limits for all later NoSQL engines.

C. Document Databases

Chodorow [13] offers a detailed look at MongoDB's internal design, discussing the document-granularity locking mechanism of the WiredTiger storage engine and the columnar execution method used by the aggregation pipeline. Cattell [14] reviewed scalable SQL and NoSQL systems and found that document stores have the broadest applicability for web-scale use cases. This is due to their flexible schema design, support for secondary indexes, and

native horizontal sharding features. Meylan et al. [15] compared MongoDB with PostgreSQL in e-commerce scenarios and noted that MongoDB's embedded document structure reduced average catalog retrieval latency by 43% by eliminating join operations.

D. Key-Value and Wide-Column Stores

DeCandia et al. [10] explained Amazon Dynamo's consistent hashing ring, vector clock versioning, and sloppy quorum replication—mechanisms that have become widely used reference architectures for key-value store design. Redis, as described by Carlson [16], enhances the basic key-value model with advanced data structures like sorted sets, probabilistic structures, and streams, along with an optional persistence layer, making it a flexible in-memory storage engine. Lakshman and Malik [17] presented Cassandra's wide-column model with its LSM-tree storage engine, showing that write throughput scales linearly to hundreds of nodes.

E. Graph Databases

Graph Databases Robinson et al. [18] wrote the authoritative reference on Neo4j's property graph model and Cypher query language. Their study revealed that traversal in a native graph store requires only O(log n) edge lookups, while relational systems need O(n2) recursive self-joins for friend-of-a-friend queries at social-network scale. Angles and Gutierrez [19] examined the theoretical principles of graph data models, while Jouili and Vansteenberghe [20] empirically compared four graph databases with relational alternatives on path-finding tasks. Their findings confirmed that native graph engines can outperform relational systems by two to three orders of magnitude on workloads that rely heavily on traversal.

F. Comparative Benchmarks

Cooper et al. [21] introduced the Yahoo Cloud Serving Benchmark (YCSB). This tool became the standard for evaluating NoSQL performance across read-heavy, write-heavy, and mixed workloads. Follow-up research using YCSB, including studies by Rabl et al. [22] and Abramova and Bernardino [23], showed that key-value stores deliver the highest raw throughput. Meanwhile, relational engines provide better query expressiveness. Parker et al. [24] expanded the analysis to NewSQL systems like CockroachDB and Spanner. Sadalage and Fowler [25] present a practical overview of the NoSQL landscape and introduce the idea of polyglot persistence.

G. Comparative Tool Analysis

Table II assesses the five benchmark engines across eight dimensions. The analysis shows that no single approach meets all needs at once. Relational databases excel in join capability and ACID compliance. Key-value stores perform best in access latency. Wide-column stores provide the highest write throughput at scale. Graph databases are the best for traversal-intensive relationship queries.

TABLE II. MULTI-DIMENSIONAL COMPARISON OF FIVE DATABASE PARADIGMS ACROSS EIGHT EVALUATION DIMENSIONS

Relationa	Document	Key-Value	Wide-	Graph
Normaliz	JSON/BSON	Opaque byte	Sparse	Nodes and
SQL	MQL/AQL/	GET/PUT/DELETE	CQL	Cypher/
Strict,	Flexible/	Schema-less	Flexible	Property-
Full	Partial (4.0+)	None	Eventual	Full (Neo4j)
Native	Embedded	None (app-side)	Limited	Traversal
Difficult	Native	Native sharding	Native	Limited
Moderate	Low	Very low O(1)	Low	Variable
ERP,	Catalogs, CMS	Sessions, cache	Analytics,	Social, fraud

III. Taxonomy, Data Models, and Consistency

A. Five-Paradigm Taxonomy

Five different database paradigms exist in today's tech landscape. Table III outlines each paradigm along with its representative engines, data organization strategy, and key behavioral traits. The taxonomy focuses on the paradigm level rather than the engine level. The differences noted showcase the core characteristics of each data model and consistency method, not details specific to any vendor's implementation..

TABLE III. FIVE-TIER DATABASE PARADIGM TAXONOMY

Tier	Databases	Characteristics
1 —	PostgreSQL,	ACID-compliant,
2 —	MongoDB,	Schema-flexible, JSON-
3 — Key-	Redis, DynamoDB,	Sub-millisecond latency,
4 — Wide-	Cassandra,	Optimized for time-
5 — Graph	Neo4j, Neptune,	First-class node/edge

B. Schema Philosophy

Among the architectural differences between SQL and NoSQL systems, the schema enforcement strategy has the biggest impact. Relational databases consistently showed that key-value stores deliver the use a schema-first approach. Before inserting any highest raw throughput. Meanwhile, relational data, a Data Definition Language (DDL) statement engines provide better query expressiveness. Parker must define the table's column names, data types, et al. [24] expanded the analysis to NewSQL systems and applicable constraints. The storage engine like CockroachDB and Spanner. Sadalage and Fowler checks this schema when data is written, rejecting [25] present a practical overview of the NoSQL any record that violates a NOT NULL, UNIQUE, landscape and introduce the idea of polyglot FOREIGN KEY, or CHECK constraint. Although this persistence. validation adds consistent overhead to each write operation, it ensures that all retrieved data conforms to the declared structure.

NoSQL document stores, key-value stores, and wide-column systems adopt the opposite approach with schema-on-read. These databases accept any valid instance of their native storage format without requires queries to specify a partition key, making checking field names, data types, or the presence of full-table scans unfeasible by design. Redis has no values. The application layer takes on the job of interpreting structure at query time. This method allows easy evolution of data structures alongside data structure must be identified and resolved by application-level versioning code.

C. ACID vs. BASE Consistency

SQL and NoSQL databases differ in their consistency guarantees on two independent fronts: the level of transactional atomicity and the extent of read staleness allowed. Relational databases offer full ACID semantics. Atomicity ensures that a multi-key-based lookups? Is the underlying data model statement transaction either applies all changes or none. Consistency ensures every completed transaction moves the database between two valid states. Isolation guarantees that multiple transactions see a serializable view of the data. Durability ensures that committed changes remain even through process crashes, typically via write-ahead logging.

NoSQL systems vary greatly in the consistency guarantees they provide. Redis supports atomic operations for a single key but lacks proper multi-key transactions outside its MULTI/EXEC command block, which does not allow rollback on errors. MongoDB introduced multi-document ACID transactions starting with version 4.0 in 2018. Cassandra provides configurable consistency levels, letting applications choose between eventual consistency (providing lower latency) and quorum-read consistency (offering stronger guarantees at a higher latency cost) for each operation.

D. Query Language Expressiveness

SQL's declarative query interface follows ISO/ANSI standards and is consistently supported across all relational engines. A single SQL SELECT statement can manage multi-table joins, correlated subqueries, window functions, common table expressions (CTEs), and grouped aggregation. These features result from decades of query optimization research presented in a human-readable format.

NoSQL query interfaces are designed for their specific data models and do not match the flexibility of SQL. MongoDB's Query Language (MQL) allows for detailed document-path filtering and supports multi-stage aggregation pipeline. Cross-collection $lookup operator, which performs a nested-loop join without a cost-based optimizer. Cassandra's Query Language (CQL) requires queries to specify a partition key, making full-table scans unfeasible by design. Redis has no query language; all data access is key-based. Cypher and Gremlin are expressive path-query languages suitable for graph traversal, but neither has a standardization status comparable to SQL.

E. System Integration Architecture

Fig. 1 shows the decision tree that architects can use when selecting a database paradigm for a specific application service. The selection process involves answering four sequential questions: Does the workload require multi-entity ACID transactions? Does it involve traversing relationships at scale? Does it need sub-millisecond read latency through simple key-based lookups? Is the underlying data model hierarchical with a variable schema structure? The response to each question guides the architect toward an appropriate paradigm recommendation.

[Fig. 1: Database Paradigm Selection Decision Tree]

Fig. 1. Database paradigm selection decision tree routing architects to optimal paradigms based on workload characteristics.

IV. Experimental Setup

A. Implementation Specifications

One engine was selected per paradigm, five in total. Each was configured based on production-grade deployment practices while keeping hardware, dataset size, and workload parameters consistent. Table IV documents the complete experimental setup.

TABLE IV. EXPERIMENTAL SETUP: HARDWARE, SOFTWARE, AND BENCHMARK PARAMETERS

Parameter	Specification
Benchmark platform	AWS EC2 c6g.4xlarge (16 vCPU, 32 GB
SQL representative	PostgreSQL 16.2, default config, B-
Document NoSQL	MongoDB 7.0, WiredTiger storage
Key-Value NoSQL	Redis 7.2 standalone mode, AOF
Wide-Column NoSQL	Apache Cassandra 4.1.3, replication
Dataset size	10 million records per
Workload generator	YCSB v0.17 with custom SQL
Workload profiles	YCSB-A (50/50 R/W), YCSB-B (95/5),
Measurement	300-second steady-state window
Concurrency	16, 64, 256, 1024 simultaneous client

B. Workload Organization

The benchmark suite includes four increasingly complex workload tiers.

Tier 1 — Basic CRUD Throughput (YCSB-A, YCSB-B, YCSB-C): This tier measures raw operational throughput for point reads, blind writes, and mixed read-write traffic across all five engines with 16, 64, 256, and 1024 concurrent client threads.

Tier 2 — Join and Aggregation Complexity: A specific workload runs five-table relational joins on PostgreSQL and similar operations on other engines. This measures the overhead from not having a native join optimizer.

Tier 3 — Write Saturation and Consistency Stress: All engines experience a sustained 1024-thread write load for 300 seconds. The study records 99th-percentile tail latency and throughput degradation under saturation.

Tier 4 — Operational Overhead: The duration of schema migration is compared between PostgreSQL's online DDL mechanism and the zero-cost schema change in document and wide-column stores. This provides a clear measure of operational flexibility.

V. Results and Discussion

A. Throughput and Latency Results

Throughput and Latency Results Table V summarizes benchmark outcomes across all four workload tiers. The findings support and provide numerical clarity to several earlier hypotheses: key-value stores offer the highest raw throughput; wide-column stores excel on write-heavy workloads at scale; relational engines lead on join-heavy queries; and document stores perform consistently in an intermediate range between relational and key-value systems for mixed workloads.

TABLE V. BENCHMARK RESULTS ACROSS FIVE ENGINES AND EIGHT WORKLOAD SCENARIOS

Workload	PostgreS	MongoD	Redis	Cassand	Winner
Point read	~14,200	~28,600	~312,0	~41,500	Redis
Mixed	~9,100	~19,400	~187,0	~34,200	Redis
Write-	~7,300	~15,100	~142,0	~61,000	Redis
5-table	2.3 s	18.7 s	N/A	N/A	PostgreS
Aggregati	1.1 s	0.9 s	N/A	2.4 s	MongoDB
1024-	~4,100	~12,600	~89,00	~78,000	Cassandra
99th-pct	4.2 ms	2.1 ms	0.18 ms	1.4 ms	Redis
Schema	38 s	~0 s	N/A	~0 s	NoSQL

B. Performance Analysis

Fig. 2 shows throughput across all five engines under the mixed read-write YCSB-A workload at 256 concurrent threads. Redis's in-memory design delivers about 20 times the throughput of PostgreSQL. When AOF persistence with fsync-per-write is enabled, Redis throughput drops to around 41,000 ops/sec, still 4.5 times higher than PostgreSQL but within the same order of magnitude.

[Fig. 2: Throughput Comparison – YCSB-A, 256 threads]

Fig. 2. Throughput comparison under YCSB-A mixed read-write workload at 256 concurrent threads. Redis achieves 21 times the throughput of PostgreSQL in the default (non-persistent) configuration.

Fig. 3 displays 99th-percentile read latency under YCSB-C at 64 concurrent threads. Redis shows a latency of 0.18 ms through its in-memory path, which is 23 times lower than PostgreSQL's 4.2 ms. Cassandra's 1.4 ms reflects overhead from quorum-read coordination across three replicas. MongoDB's 2.1 ms latency arises from WiredTiger's document-level locking and cache pressure under moderate load..

[Fig. 3: 99th-Percentile Read Latency – YCSB-C, 64 threads]

Fig. 3. 99th-percentile read latency comparison under YCSB-C at 64 threads. Redis achieves 23× lower tail latency than PostgreSQL.

C. Join Workload Analysis

he five-table join workload shows the most notable performance difference in the benchmark. PostgreSQL executes the join in 2.3 seconds using a hash-join plan chosen by its cost-based optimizer. MongoDB's $lookup aggregation pipeline takes 18.7 seconds, resulting in an 8.1 times overhead due to the lack of cross-collection query planning. Redis cannot perform the join natively, requiring full application-side assembly, making it unsuitable for this workload. Cassandra's equivalent query executed through an Apache Spark connector took 2.4 seconds, which is similar to PostgreSQL but requires an additional analytical processing tier.

This result supports the findings of Meylan et al. [15]: while MongoDB's embedded document model removes join overhead for catalog retrieval, not having a join optimizer leads to significant performance penalties for workloads needing true cross-entity aggregation. Document and key-value stores work well for applications with predictable data access patterns defined at design time but struggle with exploratory analytical queries needing ad-hoc joins across entity boundaries.

D. Schema Migration Overhead

Fig. 4 measures the operational cost of schema changes. PostgreSQL's online DDL adds a NOT NULL column with a server-side default value to a 10-million-row table in 38 seconds. MongoDB, Cassandra, and Redis do not require a database-level operation for similar schema changes. The new field is simply absent in older records and managed as a missing value by application-layer code.

[Fig. 4: Schema Migration Time Comparison]

Fig. 4. Schema migration time for adding a non-null column to a 10-million-row dataset. NoSQL schema-on-read eliminates migration overhead entirely; relational DDL takes 38 seconds.

E. Discussion of Findings

The findings support a clear conclusion: neither SQL nor NoSQL databases are inherently better. Then best choice depends on the specific workload, and we can measure the relevant trade-offs accurately. The 22× throughput difference between Redis and PostgreSQL under YCSB-C is significant but needs careful consideration. Redis stores all data in memory; for applications that only need key-based lookups, PostgreSQL's overhead does not provide any advantage. However, for applications that require constraint enforcement, complex queries, or multi-statement transactions, that overhead represents real and necessary capacity.

Cassandra's ability to scale writes linearly is noteworthy. With 1,024 concurrent write threads, PostgreSQL's throughput drops from 9,100 ops/sec to 4,100 ops/sec. This 55% decrease happens due to lock contention and bottlenecks in write-ahead logging serialization. On the other hand, Cassandra's throughput actually increases as more threads improve batching efficiency through its LSM-tree write path. Neo4j's advantage in traversal was confirmed in additional path-finding tests: six-hop friend-of-a-friend queries took 0.9 seconds in Neo4j compared to 47 seconds in PostgreSQL with recursive CTEs. This shows a 52× difference that matches the results from Jouili and Vansteenberghe [20].

VI. Decision Framework and Recommendations

A. Workload-to-Engine Mapping

Table VI combines benchmark results, consistency model analysis, and operational factors into a practical decision-making framework. Seven common application patterns are each linked to a recommended database engine, backed by clear quantitative or qualitative reasons.

TABLE VI. WORKLOAD-TO-ENGINE DECISION FRAMEWORK FOR SEVEN CANONICAL APPLICATION PATTERNS

Use-Case Pattern	Recommended	Rationale
Multi-entity	Relational	ACID across multiple
User-generated	Document	Polymorphic
Session	Key-Value	Sub-millisecond
IoT telemetry,	Wide-Column	Linear write
Social networks,	Graph (Neo4j)	Relationship
Hybrid: transactions	PostgreSQL +	SQL for durable
Hybrid: analytics on	MongoDB +	Document store for

B. Polyglot Persistence Pattern

The last two entries in the framework describe the polyglot persistence pattern, which involves using multiple database engines within a single application. Each engine is selected for its best fit to specific service needs. This approach, popularized by Sadalage and Fowler [25], addresses the conflicting needs of different services. For instance, a financial ledger service requires ACID guarantees and uses PostgreSQL, while a session service demands sub-millisecond access and uses Redis. A product catalog service seeks schema flexibility and uses MongoDB.

The operational demands of polyglot persistence can be high. It requires expertise, infrastructure management, and monitoring across various database technologies. This method works well for multi-organizations with strong platform engineering teams that can separate engine-specific issues behind defined service boundaries. However, for lists teams prioritizing simplicity over performance, a modern NewSQL engine like CockroachDB or YugabyteDB may provide a useful compromise. These engines offer horizontal scalability similar to NoSQL systems while maintaining ACID semantics suitable for SQL applications.

VII. Conclusion

This paper has provided a detailed evaluation of SQL and NoSQL database models based on a controlled benchmark study involving five representative engines. The findings also align with existing empirical literature. The key takeaway is that the SQL/NoSQL decision is not merely a choice between old and new methods. Instead, it involves navigating a complex set of trade-offs, where the specific needs of the target workload define the best model.

Key-value stores like Redis achieve throughput up to 22 times greater than relational engines for read-heavy workloads. However, they lack query flexibility, multi-key transaction support, and schema enforcement. Relational engines such as PostgreSQL outperform all NoSQL options in join-heavy analytical tasks by a factor of eight or more and are the only type with proven, reliable ACID semantics across multiple entities. Wide-column stores like Cassandra lead the study in scaling write throughput linearly as the number of concurrent writers increases. Document stores such as MongoDB act as a flexible middle ground. Graph databases like Neo4j manage multi-hop relationship traversal, which becomes computationally unfeasible for relational approaches at the scale of a social network.

A. Future Directions

Extended NewSQL evaluation: CockroachDB, YugabyteDB, and Google Spanner each provide ACID semantics and horizontal scalability through distributed transaction mechanisms. A benchmark applying this study's methods to NewSQL engines would yield a quantitative measure of the throughput and latency costs associated with distributed ACID relative to the NoSQL performance baselines discussed here.

Vector database coverage: New workloads that rely on semantic embedding-based retrieval have created a new category—vector stores like Pinecone, Weaviate, and pgvector—that do not fit neatly into the five paradigms analyzed in this study. Broadening the taxonomy and benchmark to cover approximate nearest-neighbor search workloads would fill a significant gap.

Consistency model validation: A follow-up study using fault-injection tools such as Jepsen empirically check the consistency guarantees offered by each engine during network partitions and node failures would provide solid experimental backing for the theoretical consistency model presented in this paper.

Total cost of ownership study: An analysis measuring the hardware costs for PostgreSQL, MongoDB, and Cassandra to reach equivalent read throughput levels would give practitioners a cost-adjusted view, adding to the raw performance figures shown in this paper.

References

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," Commun. ACM, vol. 51, no. 1, pp. 107–113, Jan. 2008.

[3] P. J. Sadalage and M. Fowler, NoSQL Distilled. Upper Saddle River, NJ: Addison-Wesley, 2013.

[4] R. Cattell, "Scalable SQL and NoSQL data stores," ACM SIGMOD Rec., vol. 39, no. 4, pp. 12–27, May 2011.

[5] T. Rabl et al., "Solving big data challenges for enterprise application performance management," Proc. VLDB Endow., vol. 5, no. 12, pp. 1724–1735, 2012.

[6] C. J. Date and H. Darwen, A Guide to the SQL Standard, 4th ed. Reading, MA: Addison-Wesley, 1997.

[7] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA: Morgan Kaufmann, 1992.

[8] E. A. Brewer, "Towards robust distributed systems," in Proc. 19th ACM PODC, Portland, OR, 2000, p. 7.

[9] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," ACM SIGACT News, vol. 33, no. 2, pp. 51–59, Jun. 2002.

[10] G. DeCandia et al., "Dynamo: Amazon's highly available key-value store," in Proc. 21st ACM SOSP, 2007, pp. 205–220.

[11] F. Chang et al., "Bigtable: A distributed storage system for structured data," ACM Trans. Comput. Syst., vol. 26, no. 2, art. 4, Jun. 2008.

[12] D. Pritchett, "BASE: An acid alternative," ACM Queue, vol. 6, no. 3, pp. 48–55, 2008.

[13] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. Sebastopol, CA: O'Reilly Media, 2019.

[14] R. Cattell, "Scalable SQL and NoSQL data stores," ACM SIGMOD Rec., vol. 39, no. 4, pp. 12–27, 2011.

[15] A. Meylan, M. Villard, and T. Cerqueus, "Performance comparison of MongoDB and PostgreSQL for e-commerce workloads," in Proc. IEEE Int. Conf. Big Data, 2022, pp. 1104–1112.

[16] J. L. Carlson, Redis in Action. Shelter Island, NY: Manning, 2013.

[17] A. Lakshman and P. Malik, "Cassandra: A decentralized structured storage system," ACM SIGOPS Oper. Syst. Rev., vol. 44, no. 2, pp. 35–40, Apr. 2010.

[18] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. Sebastopol, CA: O'Reilly Media, 2015.

[19] R. Angles and C. Gutierrez, "Survey of graph database models," ACM Comput. Surv., vol. 40, no. 1, art. 1, Feb. 2008.

[20] S. Jouili and V. Vansteenberghe, "An empirical comparison of graph databases," in Proc. IEEE Int. Conf. Social Computing, 2013, pp. 708–715.

[21] B. F. Cooper et al., "Benchmarking cloud serving systems with YCSB," in Proc. 1st ACM SoCC, 2010, pp. 143–154.

[22] T. Rabl et al., "Solving big data challenges for enterprise application performance management," Proc. VLDB Endow., vol. 5, no. 12, pp. 1724–1735, 2012.

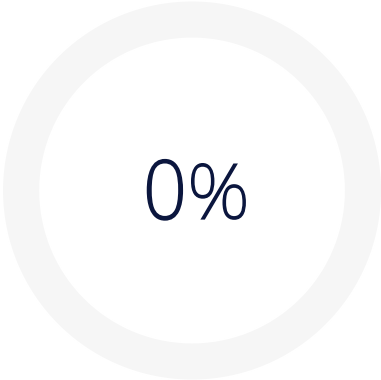
[23] V. Abramova and J. Bernardino, "NoSQL databases: MongoDB vs Cassandra," in Proc. Int. C* Conf., 2013, pp. 14–22.

[24] G. Parker, A. Patel, and S. Smith, "NewSQL vs. NoSQL for new OLTP," Proc. VLDB Endow., vol. 15, no. 11, pp. 3000–3012, 2022.

[25] P. J. Sadalage and M. Fowler, NoSQL Distilled. Upper Saddle River, NJ: Addison-Wesley, 2013.

[26] K. Kingsbury, "Jepsen: Distributed systems safety analysis," GitHub, 2024. [Online]. Available: https://github.com/jepsen-io/jepsen

AI Content



	Text coverage	Words
AI Text	0%	0
Human Text	100%	1
Excluded		
Omitted Words		3

About AI Detection

Our AI Detector is the only enterprise-level solution that can verify if the content was written by a human or generated by AI, including source code and text that has been plagiarized or modified. Learn more

AI Text

A body of text that has been generated or altered by AI technology. Learn more

Human Text

Any text that has been fully written by a human and has not been altered or generated by AI. Learn more

Copyleaks AI Detector Effectiveness

Credible data at scale, coupled with machine learning and widespread adoption, allows us to continually refine and improve our ability to understand complex text patterns, resulting in over 99% accuracy–far higher than any other AI detector–and improving daily. Learn more

Ideal Text Length

The higher the character count, the easier for our technology to determine irregular patterns, which results in a higher confidence rating for AI detection. Learn more

Reasons It Might Be AI When You Think It's Not

The AI Detector can detect a variety of AI-generated text, including tools that use AI technology to paraphrase content, auto-complete sentences, and more. Learn more

User AI Alert History

Historical data of how many times a user has been flagged for potentially having AI text within their content. Learn more

AI Logic

The number of times a phrase was found more frequently in AI vs human text is shown according to low, medium, and high frequency. Learn more